

---

## # 🏛️ The Ultimate "Giant-Killer" Manga Platform Architecture Plan

### ## 📌 1. Executive Summary

This platform is built as a **"Decoupled, Queue-Driven, Edge-Cached Machine"**.

Instead of a traditional server that processes requests in real-time (which crashes under load, gets IP-banned, and makes users wait), this system separates data scraping, image processing, and user delivery into three isolated tiers using an **"asynchronous job queue"**.

**Result:** A platform capable of surviving massive traffic spikes, delivering instant page loads, preventing mobile browser crashes via memory-safe image slicing, handling upstream API failures gracefully, requiring **"absolutely zero credit cards"**, and costing exactly \$0.00 forever.

---

### ## 🛠️ 2. The Tech Stack

- \* **Frontend:** **"Next.js 15"** (App Router, React 19, ISR) hosted on **"Cloudflare Pages"** via OpenNext.

- \* **Database & Auth:** **"Supabase"** (PostgreSQL). 500MB DB, 50k MAU, built-in Auth ('@supabase/ssr'), Row Level Security.

- \* **Image Storage:** **"Cloudflare R2"**. Zero egress fees. First 10GB free.

- \* **Edge API Layer:** **"Cloudflare Workers"**. Caches JSON responses at the edge to protect the Supabase database.

- \* **Backend Compute Engine:** **"Hugging Face Spaces (Docker)"**. Runs a background daemon (via Bun) for image processing. 2 vCPU, 16GB RAM. No credit card required.

- \* **Cron Scraper:** **"GitHub Actions"**. 2,000 free minutes/month. Runs hourly to ingest new chapters.

- \* **APIs Consumed:** Jikan API (Metadata), FireFly API (Primary), MangaHook API (Failover).

---

### ## 🧠 3. The "Giant Platform" Strategies Applied

1. **"Asynchronous State Machine:"** The Cloudflare Worker *never* waits for the backend to process images. It only serves content marked as `'READY'` in the database. New chapters are `'QUEUED'` and processed in the background.

2. **"Provider Adapter Pattern:"** FireFly and MangaHook APIs are hidden behind a common interface. If one API dies, the system fails over to the other without rewriting frontend code.

3. **"Exponential Backoff & Dead Letter Queue:"** If an upstream API fails, the worker retries 3 times with exponential delays. If it still fails, it is moved to a Dead Letter Queue for admin review.

4. **\*\*Manhwa Slicing (Mobile Crash Prevention):\*\*** The backend uses `sharp` to slice any image taller than 1500px into chunks. The frontend seamlessly stitches them back together, keeping mobile memory usage near zero.
5. **\*\*PostgreSQL Full-Text Search:\*\*** Utilizing Supabase's built-in `tsvector` to search titles, authors, and descriptions without needing a separate paid search engine.
6. **\*\*Zero IP Bans:\*\*** Users never communicate with upstream APIs. The Hugging Face worker fetches and caches images to R2. The frontend only serves R2 links.

---

## ## 4. Database Schema (Supabase / PostgreSQL)

\*Integrates Job Queue States, Full-Text Search, and Dead Letter Queue.\*

```
```sql
-- Manga Metadata with Full-Text Search
CREATE TABLE manga (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  mal_id INT UNIQUE,
  title TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  alt_titles TEXT[],
  author TEXT,
  description TEXT,
  cover_r2_key TEXT,
  status TEXT DEFAULT 'ongoing',
  search_vector tsvector GENERATED ALWAYS AS (to_tsvector('english', coalesce(title, '') || ' ' || coalesce(description, '') || ' ' || coalesce(author, ''))) STORED,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE INDEX manga_search_idx ON manga USING gin(search_vector);

-- Chapters with Job Queue States
CREATE TABLE chapters (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  manga_id UUID REFERENCES manga(id) ON DELETE CASCADE,
  chapter_number FLOAT NOT NULL,
  external_chapter_id TEXT,
  provider TEXT DEFAULT 'firefly',
  job_status TEXT DEFAULT 'DISCOVERED', -- DISCOVERED, QUEUED, PROCESSING,
  READY, FAILED
  retry_count INT DEFAULT 0,
  last_error TEXT,
  released_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
)
```

```

);

-- Normalized Pages (Supports image slicing)
CREATE TABLE pages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chapter_id UUID REFERENCES chapters(id) ON DELETE CASCADE,
  page_number INT NOT NULL,
  upstream_url TEXT,
  r2_keys TEXT[] DEFAULT '{}
);

-- User Tracking
CREATE TABLE reading_progress (
  user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  manga_id UUID REFERENCES manga(id) ON DELETE CASCADE,
  chapter_id UUID REFERENCES chapters(id) ON DELETE CASCADE,
  page_number INT DEFAULT 0,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (user_id, manga_id)
);

-- Dead Letter Queue Table (For Admin Review)
CREATE TABLE dead_letter_queue (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  chapter_id UUID REFERENCES chapters(id) ON DELETE CASCADE,
  error_message TEXT NOT NULL,
  failed_at TIMESTAMPTZ DEFAULT NOW(),
  resolved BOOLEAN DEFAULT FALSE
);
...

```

---

## ## 🚧 5. Multi-Source Provider Adapter Layer

\*Supports interchangeable providers so you can add/remove APIs without redesigning the system.\*

```

``typescript
// providers/AdapterInterface.ts
export interface MangaProvider {
  fetchLatestManga(page: number): Promise<any>;
  fetchChapterPages(chapterId: string): Promise<string[]>;
}

```

```
// providers/FireFlyAdapter.ts
export class FireFlyAdapter implements MangaProvider {
  async fetchChapterPages(chapterId: string): Promise<string[]> {
    const res = await fetch(`http://localhost:3000/api/fetch-chapter?id=${chapterId}`);
    const data = await res.json();
    return data.images;
  }
}
...
---
```

## ## 🐙 6. The Hugging Face Worker (Bun + Docker)

\*Runs as a background Docker container. Features Exponential Backoff, Dead Letter Queue, and a dummy HTTP server to prevent sleeping.\*

```
```dockerfile
# Dockerfile
FROM oven/bun:1.1
```

```
WORKDIR /app
COPY package.json bun.lockb ./
RUN bun install --production
COPY . .
```

```
EXPOSE 7860
CMD ["bun", "run", "worker.ts"]
...

```

```
```typescript
// worker.ts - Runs via Bun on Hugging Face
import { createClient } from '@supabase/supabase-js';
import sharp from 'sharp';
import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3';
import http from 'http';
```

```
// 1. Dummy HTTP server to keep Hugging Face Space awake (ping via UptimeRobot)
http.createServer((req, res) => res.writeHead(200).end('Worker is alive')).listen(7860);
```

```
const supabase = createClient(process.env.SUPABASE_URL!, process.env.SUPABASE_KEY!);
const r2 = new S3Client({
  region: 'auto',
  endpoint: process.env.R2_ENDPOINT!,
```

```
    credentials: { accessKeyId: process.env.R2_KEY!, secretAccessKey:
process.env.R2_SECRET! }
});
```

```
const MAX_RETRIES = 3;
const BACKOFF_MS = 60000;
const SLICE_HEIGHT = 1500;
```

```
async function processQueue() {
  const { data: chapter } = await supabase
    .from('chapters')
    .select('id, external_chapter_id, retry_count')
    .eq('job_status', 'QUEUED')
    .limit(1)
    .single();
```

```
  if (!chapter) {
    setTimeout(processQueue, 10000);
    return;
  }
```

```
  await supabase.from('chapters').update({ job_status: 'PROCESSING' }).eq('id', chapter.id);
```

```
  try {
    // Fetch upstream URLs (Adapter logic here)
    const pagesData = []; // Replace with actual adapter call
```

```
    for (const [index, page] of pagesData.entries()) {
      const imgRes = await fetch(page.url);
      const buffer = Buffer.from(await imgRes.arrayBuffer());
```

```
      const metadata = await sharp(buffer).metadata();
      const slices = Math.ceil(metadata.height / SLICE_HEIGHT);
      const r2Keys = [];
```

```
      for (let i = 0; i < slices; i++) {
        const sliceBuffer = await sharp(buffer)
          .extract({
            top: i * SLICE_HEIGHT,
            left: 0,
            width: metadata.width,
            height: Math.min(SLICE_HEIGHT, metadata.height - (i * SLICE_HEIGHT))
          })
          .toFormat('webp', { quality: 70 })
```

```

        .toBuffer();

        const r2Key = `manga/${chapter.id}/${page.id || index}_${i}.webp`;
        await r2.send(new PutObjectCommand({
            Bucket: 'manga-images',
            Key: r2Key,
            Body: sliceBuffer,
        }));
        r2Keys.push(r2Key);
    }
    await supabase.from('pages').update({ r2_keys: r2Keys }).eq('id', page.id);
}

await supabase.from('chapters').update({
    job_status: 'READY',
    retry_count: 0,
    last_error: null
}).eq('id', chapter.id);

} catch (err: any) {
    console.error(`Chapter ${chapter.id} failed:`, err.message);

    if (chapter.retry_count >= MAX_RETRIES) {
        await supabase.from('dead_letter_queue').insert({
            chapter_id: chapter.id,
            error_message: err.message
        });
        await supabase.from('chapters').update({
            job_status: 'FAILED',
            last_error: err.message
        }).eq('id', chapter.id);
    } else {
        const nextRetryTime = new Date(Date.now() + (BACKOFF_MS * Math.pow(2,
chapter.retry_count)));
        await supabase.from('chapters').update({
            job_status: 'QUEUED',
            retry_count: chapter.retry_count + 1,
            last_error: err.message,
            updated_at: nextRetryTime
        }).eq('id', chapter.id);
    }
}

processQueue();

```

```
}
```

```
processQueue();
```

```
...
```

```
---
```

## ☁️ 7. The Edge API Layer (Cloudflare Worker)

\*Serves only READY content. Caches JSON at the edge to protect Supabase.\*

```
``typescript
```

```
// cloudflare-worker-api.ts
```

```
export default {
```

```
  async fetch(request: Request, env: any, ctx: ExecutionContext) {
```

```
    const url = new URL(request.url);
```

```
    const cacheKey = new Request(url.toString(), request);
```

```
    const cache = caches.default;
```

```
    let response = await cache.match(cacheKey);
```

```
    if (response) return response;
```

```
    const chapterId = url.searchParams.get('chapterId');
```

```
    if (!chapterId) return new Response('Missing chapterId', { status: 400 });
```

```
    const supabaseRes = await
```

```
    fetch(`${env.SUPABASE_URL}/rest/v1/chapters?id=eq.${chapterId}&job_status=eq.READY&select=*,pages(*)`, {
```

```
      headers: {
```

```
        'apikey': env.SUPABASE_ANON_KEY,
```

```
        'Authorization': `Bearer ${env.SUPABASE_ANON_KEY}`
```

```
      }
```

```
    });
```

```
    const data = await supabaseRes.json();
```

```
    if (!data || data.length === 0) {
```

```
      return new Response(JSON.stringify({ error: "Chapter not ready or not found" }), {
```

```
        status: 404,
```

```
        headers: { 'Content-Type': 'application/json' }
```

```
      });
```

```
    }
```

```
    response = new Response(JSON.stringify(data), {
```

```
      headers: {
```

```

    'Content-Type': 'application/json',
    'Cache-Control': 'public, max-age=3600'
  }
});

ctx.waitUntil(cache.put(cacheKey, response.clone()));
return response;
}
};
...

```

---

## ## 🛡️ 8. Free-Tier Capacity Limits & Upgrade Triggers

| Service            | Free Limit       | First Bottleneck | Upgrade Trigger                                                                                                      |
|--------------------|------------------|------------------|----------------------------------------------------------------------------------------------------------------------|
| Cloudflare R2      | 10 GB Storage    | Storage space    | When 10GB is full. *Action: Enable the 30-day eviction script for old chapters, or upgrade to R2 paid (\$0.015/GB).* |
| Supabase DB        | 500 MB           | Database Size    | 500MB holds ~50,000 chapters. *Action: Upgrade to Supabase Pro (\$25/mo).*                                           |
| Hugging Face       | 2 vCPU, 16GB RAM | Image Processing | If >50 chapters drop per hour. *Action: Deploy a second Hugging Face Space to double throughput.*                    |
| GitHub Actions     | 2,000 min/month  | Cron Scraper     | 1 min script x 24 hours x 30 days = 720 mins. Plenty of room.                                                        |
| Cloudflare Workers | 100k req/day     | API Caching      | Bypassed by ISR. Only breaks if 100k *uncached* chapters are accessed daily.                                         |

### \*\*Health Checks & Monitoring.\*\*

- \* **Docker Auto-Restart:** Hugging Face automatically restarts crashed containers.
- \* **UptimeRobot:** Pings the Hugging Face Space port (7860) every 10 minutes to prevent the free tier from sleeping.
- \* **Dead Letter Queue UI:** A protected `/admin/dlq` route in Next.js queries the `dead\_letter\_queue` table for manual review of permanently failed chapters.

---

## ## 💰 9. Monetization Strategy

Aggregator platforms using scraped APIs cannot use Google AdSense. You must use alternative networks.

- Primary Network:** Adsterra (Native Banners) or Monetag (Smart Links).
- Placement Strategy:**
  - \* **Desktop:** 728x90 banner above the reader, 300x600 sticky sidebar.

\* \*Mobile:\* 320x50 sticky footer ad.

3. **Anti-Ban Protection:** Implement **Cloudflare Turnstile** (free) on your search bar to stop bots from clicking ads. Enable Cloudflare Bot Fight Mode.

---

## ## 🚀 10. Phase-by-Phase AI Implementation Prompts

Here are the highly engineered, professional prompts designed to be fed into Claude 3.5 Sonnet or GPT-4o to generate the exact codebase.

### ### 🗄️ Phase 1 AI Prompt: Database Foundation

> **System Role:** You are a Staff-Level Database Architect specializing in PostgreSQL and Supabase.

>

> **Context:** I am building a high-throughput Manga Aggregator platform. The system relies on an asynchronous job queue pattern where chapters are scraped, placed in a queue, processed (downloaded, compressed, sliced), and finally marked as READY for user consumption.

>

> **Task:** Generate the complete Supabase SQL migration script. The script must include:

> 1. **Tables:** `manga`, `chapters`, `pages`, `reading\_progress`, `dead\_letter\_queue`.

> 2. **Data Integrity:** Use `UUID` primary keys, proper `ON DELETE CASCADE` for foreign keys, and strict `NOT NULL` constraints.

> 3. **Job Queue Fields:** `chapters` table needs `job\_status` (ENUM: DISCOVERED, QUEUED, PROCESSING, READY, FAILED), `retry\_count`, `last\_error`, `provider`.

> 4. **Search:** `tsvector` generated column on `manga` combining `title`, `alt\_titles`, `author`, `description`. GIN index.

> 5. **Security (RLS):** Enable RLS. Users `SELECT` only `READY` chapters. Users manage only their own `reading\_progress`. `dead\_letter\_queue` is admin-only (`service\_role`).

>

> **Code Quality:** Raw SQL. Comment on indexing strategies. Safe ENUM recreation.

### ### 🖥️ Phase 2 AI Prompt: Backend Engine & Docker Worker

> **System Role:** You are a Senior Backend Engineer specializing in Bun, Docker, and queue-driven architectures.

>

> **Context:** We need a backend application that runs in a Hugging Face Docker Space. It must poll Supabase for `job\_status = 'QUEUED'`, fetch upstream manga images via an Adapter pattern, process them using `sharp` (compress to WebP, slice >1500px height), and upload to Cloudflare R2.

>

> **Task:** Generate the Bun-compatible TypeScript codebase:

> 1. `providers/AdapterInterface.ts` & `providers/FireFlyAdapter.ts`.

> 2. `r2Client.ts`: AWS SDK v3 configured for R2.

> 3. `worker.ts`: The main async daemon loop. Include a dummy `http.createServer` listening on port 7860 to keep the container awake. Implement exponential backoff (base 60s) and dead-letter queue logic for failures.

> 4. `Dockerfile`: Use `oven/bun:1.1` base image. Install `sharp` dependencies. Expose 7860.

> 5. `.github/workflows/scrapper.yml`: A GitHub Action running hourly to fetch new chapters and insert them as `QUEUED`.

>

> **Code Quality:** Strict TypeScript. Memory-efficient streams/buffers. Robust try/catch.

#### ### 🏠 Phase 3 AI Prompt: Edge API & Caching

> **System Role:** You are a Cloudflare Edge Engineer specializing in Workers and cache optimization.

>

> **Context:** Our Next.js frontend needs to fetch chapter data. To protect Supabase, we need a Cloudflare Worker to act as an API proxy.

>

> **Task:** Write `worker.ts` in TypeScript:

> 1. Intercept `/api/chapter/:id`.

> 2. Check Cloudflare Cache API first (` caches.default `). Return instantly if hit.

> 3. If miss, query Supabase REST API for `job\_status=eq.READY`.

> 4. Return 404 if not ready. Save successful response to Edge Cache (` max-age=3600 `).

>

> **Code Quality:** Strict request validation (block non-GET). Basic IP rate-limiting (60 req/min, return 429). Zero external dependencies.

#### ### 🏠 Phase 4 AI Prompt: Frontend & Reader UI

> **System Role:** You are a Frontend Architect specializing in Next.js 15 (App Router), React 19, and Core Web Vitals.

>

> **Context:** We have a Cloudflare Worker API serving sliced WebP image URLs (` pages.r2\_keys ` array). We need to build the frontend.

>

> **Task:** Generate Next.js 15 App Router code:

> 1. `app/layout.tsx`: Root layout with `@supabase/ssr` Auth Provider and PWA meta tags.

> 2. `app/search/page.tsx`: Server component using Supabase `tsvector` for `?q=` queries.

> 3. `app/chapter/[id]/page.tsx`: Manga reader. Fetch from Worker API. `export const revalidate = 3600`. Map over `r2\_keys` and render in a continuous `flex flex-col` for seamless Manhwa reading. Lazy load and prefetch next 2 pages.

>

> **Code Quality:** Tailwind CSS. Strict TS interfaces. Optimize LCP/CLS (explicit image aspect ratios).

#### ### 🏠 Phase 5 AI Prompt: Monetization & Deployment

> **System Role:** You are a DevOps and Security Engineer specializing in Cloudflare, GitHub Actions, and web monetization.

>

> **Context:** Deploying Next.js to Cloudflare Pages, deploying the Worker, and integrating Adsterra securely.

>

> **Task:** Provide:

> 1. `app/components/AdScript.tsx`: `next/script` with `strategy="lazyOnload"` for Adsterra.

> 2. `app/components/Turnstile.tsx`: Cloudflare Turnstile integration for the search page.

> 3. `.github/workflows/deploy.yml`: CI/CD for `main` push. `npm ci`, `tsc --noEmit`, `npx @opennextjs/cloudflare`, deploy to Pages.

>

> **Code Quality:** Secure environment variables. Caching for `node\_modules`. Idempotent scripts.